



University of Massachusetts Amherst

Department of Microbiology¹

Department of Electrical and Computer Engineering²

.vtk Processing & 3D Rendering

Notebook Documentation

Version 1.0 · May 2026

Prepared by: Cooper Richman²

Supervisors

Dr. Sang Hyun Lee¹

Dr. Bill Leonard²

Contents

1	Introduction	1
1.1	Key Capabilities	1
1.2	Dependencies	2
2	Notebook Cells Overview	3
2.1	Cell 1: View a Single <code>.vtk</code> File	3
2.2	Cell 2: Batch Clean <code>.vtk</code> Files (Preview Only)	3
2.3	Cell 3: Batch Clean and Save <code>.vtk</code> Files	4
2.4	Cell 4: View a Cleaned <code>.vtk</code> File with Metadata	4
3	Typical Workflow	5
4	Limitations & Notes	6
5	Change Log	7

Chapter 1

Introduction

The **.vtk Processing & 3D Rendering** notebook (`.vtk_Processing.ipynb`) is a collection of Python cells for loading, cleaning, and visualizing triangulated surface meshes stored in VTK format. These meshes represent mycelial fungal growth reconstructed from microscopy data. The notebook provides tools for welding disconnected mesh geometry, isolating the largest connected component, and rendering results as interactive or static 3D images.

The processed and cleaned `.vtk` files produced by this notebook serve as the primary input for the downstream 2D and 3D skeletonization notebooks.

1.1 Key Capabilities

- Interactive 3D viewing of individual `.vtk` files via PyVista.
- Batch cleaning of raw `.vtk` files using configurable vertex welding tolerance.
- Extraction of the largest connected mesh component to remove noise and disconnected fragments.
- Side-by-side before/after rendering of original versus cleaned meshes.
- Saving cleaned meshes to a structured output directory for use in subsequent processing steps.
- Static off-screen rendering and metadata inspection of individual meshes.

1.2 Dependencies

Package	Install command	Role
pyvista	<code>pip install pyvista</code>	Loading, cleaning, and rendering .vtk meshes
matplotlib	<code>pip install matplotlib</code>	Displaying static screenshots and side-by-side comparisons
pathlib	(bundled with Python)	Cross-platform path handling

Chapter 2

Notebook Cells Overview

Each cell in this notebook can be run independently. They are ordered roughly from simplest (view a single file) to most complete (batch-clean and save all files).

2.1 Cell 1: View a Single `.vtk` File

Loads one `.vtk` file and displays it in an interactive 3D PyVista window. The mesh surface is rendered with lighting, a white background, and axis indicators. Grid labels are suppressed for clarity.

Key parameter: `vtk_path` — the filepath to the `.vtk` file to be viewed.
Change this to point to any other `.vtk` file.

2.2 Cell 2: Batch Clean `.vtk` Files (Preview Only)

Iterates over all `.vtk` files in a specified input directory, applies vertex welding and largest-component extraction, and displays a Matplotlib side-by-side comparison of the original and cleaned mesh. Results are **not saved** in this cell; it is intended for previewing the effect of different tolerance values before committing to a save.

Parameter	Default	Description
<code>tolerance</code>	1.5	Vertex merging radius. Vertices within this distance are welded together.
<code>window_size</code>	(400, 400)	Resolution of the off-screen rendering for the preview images.
<code>vtk_dir</code>	(user-defined path)	Folder containing the raw <code>.vtk</code> files to process.

2.3 Cell 3: Batch Clean and Save `.vtk` Files

Performs the same cleaning pipeline as Cell 2 but saves each cleaned mesh to a new output directory. The output folder name encodes the tolerance value used (e.g., `Cleaned VTK Data.Tol=1.5`), making it easy to compare results from different tolerance settings.

Output filenames follow the pattern:

```
<original_stem>_cleaned_tol<tolerance>.vtk
```

Do Not Overwrite Raw Data

Ensure `vtk_dir` (input) and `output_dir` are different directories. The script creates the output directory automatically if it does not exist, but it will not prevent overwriting if the paths are the same.

2.4 Cell 4: View a Cleaned `.vtk` File with Metadata

Loads a single cleaned `.vtk` file, prints mesh metadata (point count, cell count, attached data arrays), and renders a static isometric screenshot using PyVista's off-screen renderer.

Chapter 3

Typical Workflow

- (i) **Inspect a raw file.** Use Cell 1 to load a single raw `.vtk` and verify it loads correctly and looks reasonable in the interactive window.
- (ii) **Tune the tolerance.** Use Cell 2 to batch-preview cleaning results across all files. Adjust `tolerance` until disconnected faces are properly welded without over-merging distinct structures.
- (iii) **Save cleaned meshes.** Run Cell 3 with the chosen tolerance to write cleaned `.vtk` files to the output directory.
- (iv) **Verify a cleaned file.** Use Cell 4 to open a saved cleaned mesh and confirm the metadata and visual appearance are correct. The cleaned files are then ready for use in the skeletonization notebooks.

Chapter 4

Limitations & Notes

Tolerance Sensitivity

The vertex welding tolerance directly affects mesh topology. A value that is too low may leave the mesh fragmented; a value that is too high may merge distinct structures. The default of 1.5 was chosen empirically for the available dataset and may need adjustment for other imaging conditions or resolutions.

Single Connected Component Only

The cleaning step retains only the **largest** connected component. If the biological structure of interest is not the largest component (e.g., because a fragment of debris is larger), the result will be incorrect. Visually inspect outputs using Cell 4.

Interactive windows: Cells that call `plotter.show()` or `voxel_mesh.show()` open a VTK window that must be closed before the notebook cell finishes executing.

Chapter 5

Change Log

- **February 9, 2026:** File created from parts of earlier notebooks, consolidated in one place. (C. Richman)
- **May 20, 2026:** Tidied up for final repository. (C. Richman)